Short-Burst Optimization with Calibrated Chains
Composing Burst Optimization and Distributional Correctness

G.12 — Ensemble Methods / Composable Search

Gio Della-Libera

Apportionment Research Group

giodl@microsoft.com

May 2026

Abstract

Short-Burst (G.6) is a meta-algorithm: it runs n_{bursts} short Markov chains and selects the endpoint with the best edge-cut. The underlying chain type is a parameter. We implement two new chain variants—ShortBurstForest and ShortBurstMergeSplit—by composing Short-Burst with ForestRecom (G.9) and MergeSplit (G.10) chains, respectively. This pairing delivers two properties simultaneously: compactness optimization (the contribution of the Short-Burst wrapper) and approximate distributional correctness (the contribution of the ForestRecom and MergeSplit chains). We evaluate all three Short-Burst variants—standard ReCom-based, ShortBurstForest, and ShortBurstMergeSplit—on North Carolina ($k = 14$), Wisconsin ($k = 8$), and Texas ($k = 38$) at equal compute budget (1000 total steps per run). The key empirical finding is that standard ShortBurst achieves comparable or slightly better final edge-cut than the calibrated variants because its higher per-burst acceptance rate ($\approx 80\%$) explores more plans per burst; ForestRecom and MergeSplit spend compute

on the Metropolis–Hastings ratio computation that is unnecessary for pure compactness optimization. However, ShortBurstForest produces better *ensemble coverage* near good plans because each accepted step is an approximately correct sample from the stationary distribution. We introduce the *per-burst acceptance rate* diagnostic, recorded in the audit manifest, which quantifies the effective burst length for each chain type. The three variants are available via `-search short-burst-forest` and `-search short-burst-merge-split`.

Contents

1. Introduction

1.1. Short-Burst as a Meta-Algorithm

Short-Burst [Cannon et al., 2022, Della-Libera, 2026b] is a meta-algorithm for compactness optimization in redistricting. It runs n_{bursts} short Markov chains of length ℓ_{burst}, recording only the endpoint of each chain, then selects the endpoint with the best (lowest) edge-cut. The inner chain restarts from the previous winner after each burst, so the trajectory progresses by finding successively better plans—a form of iterated local search embedded in a Markov chain.

The critical observation is that *the inner chain type is a parameter.* Cannon et al. [2022] demonstrate ShortBurst with a ReCom inner chain, but there is nothing in the Short-Burst framework that requires ReCom. Any ergodic Markov chain on the space of valid redistricting plans can serve as the inner chain. Swapping the inner chain type changes the distributional properties of the endpoint without altering the burst-selection logic.

1.2. Why Chain Type Matters

The standard ReCom chain [DeFord et al., 2021] is not reversible with respect to any simple target distribution. It proposes new plans at high acceptance rates (60–80%) but does not correct for the asymmetry in proposal probabilities. For pure compactness optimization this is irrelevant—we want good plans, not samples from a specific distribution. But for ensemble sampling, the lack of distributional correctness means the endpoint distribution of a ReCom-based ShortBurst chain is biased toward whatever plans ReCom tends to overproduce.

ForestRecom [McCartan and Imai, 2020, Della-Libera, 2026c] and MergeSplit [Janson et al., 2022, Della-Libera, 2026a] both apply a Metropolis–Hastings correction to the spanning-tree proposal, making their stationary distributions approximately uniform over valid plans (subject to the population-balance constraint). When used as ShortBurst inner chains, this property carries over to the burst endpoints: the selected endpoint is not merely the best plan the (biased) chain encountered, but the best plan from a set of approximately-correctly-distributed endpoints.

This paper studies what happens when we compose ShortBurst with ForestRecom and MergeSplit: we obtain ShortBurstForest and ShortBurstMergeSplit.

1.3. Three Contributions

1. **Implementations.** We give the complete specifications of ShortBurstForest and ShortBurstMergeSplit as implemented in `redist-cli`, including the deterministic seed schedule with chain-type-specific prefixes and the two-RNG step protocol required by ForestRecom and MergeSplit (Section 3).

2. **Comparative evaluation.** We compare standard ShortBurst (G.6), ShortBurst-Forest, and ShortBurstMergeSplit on North Carolina ($k = 14$), Wisconsin ($k = 8$), and Texas ($k = 38$) at equal compute budget (1000 total steps). The key finding is that at equal budget, standard ShortBurst achieves comparable or slightly better final edge-cut because its higher acceptance rate explores more plans per burst (Section 4).

3. **Per-burst acceptance rate diagnostic.** We introduce α_{burst}, the fraction of burst steps that are accepted within a given burst. This diagnostic is recorded in the audit manifest and quantifies the effective burst length for each chain type. Lower α_{burst} means fewer accepted plans per burst, so the nominal burst length ℓ_{burst} overstates the effective exploration per burst (Section 3).

1.4. Paper Structure

Section 2 reviews the three chain types—ShortBurst, ForestRecom, and MergeSplit—and the seed architecture. Section 3 gives the formal pseudocode for ShortBurstForest and ShortBurstMergeSplit and defines the per-burst acceptance rate diagnostic. Section 4 reports the empirical comparison on NC, WI, and TX and explains the acceptance-rate mechanism behind the results. Section 5 gives usage guidance for practitioners. Section 6 concludes.

2. Background

2.1. Short-Burst (G.6)

Short-Burst [Cannon et al., 2022] resolves the tension between mixing (long chains) and optimisation (short chains) by running n_{bursts} short chains of length ℓ_{burst}, keeping only the endpoints, and selecting the best by edge-cut. Each burst restarts from the previous best endpoint, producing a progressive search that concentrates probability mass in the burst neighbourhood of each successive plan.

In the G.6 implementation, the inner chain is ReCom [DeFord et al., 2021]: merge two adjacent districts, sample a random spanning tree, cut at the balance point. ReCom accepts at 60–80% on typical census-tract graphs.

Seeding. For burst i with base seed b:

$$s^{(i)} = \mathrm{SHA256}\Big(\texttt{"SB\_CHAIN\_"} \,\|\, i^{\mathrm{le64}} \,\|\, \texttt{"\_"} \,\|\, b^{\mathrm{le64}}\Big)\,[0{:}8] \text{ as } u64.$$

Full reproducibility follows: any burst trajectory is recoverable from b alone.

2.2. Forest ReCom (G.9)

ForestRecom [McCartan and Imai, 2020, Della-Libera, 2026c] uses the same proposal as ReCom (merge two adjacent districts, sample a random spanning tree, cut at the balance point) but adds a Metropolis–Hastings correction:

$$\alpha_{\mathrm{FR}}(\pi \to \pi') = \min\left(1,\ \frac{Q(\pi' \to \pi)}{Q(\pi \to \pi')}\right).$$

Computing $Q(\pi' \to \pi)$ requires a *reverse RNG* stream in addition to the forward RNG used to sample the proposal. The correction makes the stationary distribution approximately uniform over valid plans; acceptance rate is $\approx$ 40–50%.

2.3. Merge-Split (G.10)

MergeSplit [Janson et al., 2022, Della-Libera, 2026a] applies the same merge-and-resample proposal but computes the MH ratio via a single spanning-tree-count ratio rather than the two-tree reverse evaluation used by ForestRecom. This makes MergeSplit cheaper per step while still providing a distributional correction; acceptance rate is $\approx$ 55–65%.

2.4. The Chain-Type Composition

All three chains—ReCom, ForestRecom, and MergeSplit—operate on the same state space (valid k-district plans of the census-tract adjacency graph) and propose moves via the same merge-and-resample operation. They differ only in their acceptance criterion and the RNG resources required per step.

This structural similarity makes the chain type a clean parameter for the ShortBurst wrapper: swapping the inner chain from ReCom to ForestRecom or MergeSplit changes the per-step cost and acceptance rate, but leaves the burst structure, the endpoint-selection logic, and the seed architecture intact. The G.12 implementations exploit this directly: ShortBurstForest and ShortBurstMergeSplit reuse the ShortBurst burst loop, substituting the chain type.

3. Algorithm

3.1. ShortBurstForest

ShortBurstForest is the ShortBurst meta-algorithm with ForestRecom as the inner chain. Algorithm 1 gives the complete specification.

Two RNGs per step. Each ForestRecom step requires two independent RNG streams: the *forward RNG* (to sample the proposed spanning tree and cut) and the *reverse RNG* (to evaluate the reverse proposal probability for the Metropolis–Hastings ratio). Both are derived deterministically from the burst seed s_i and the step index t using the `SBF_FWD_` and `SBF_REV_` prefixes. This ensures that any burst trajectory can be independently reproduced by any verifier who holds the base seed b and the burst parameters.

Endpoint restart. At the end of each burst, π_{cur} is updated to `chain.assignment`, the endpoint of the current burst. This is unconditional: the burst endpoint always becomes the next starting plan, regardless of whether it has lower or higher edge-cut than the previous

Algorithm 1 SHORTBURSTFOREST

Require: Burst length ℓ, burst count n, percentile $p \in [0, 1]$, base seed b
Ensure: Selected plan π^*

$\pi_{\text{cur}} \leftarrow$ METIS_plan(b) ▷ initial plan from METIS
endpoints $\leftarrow$ []
for $i \leftarrow 0$ **to** $n - 1$ **do**
  $s_i \leftarrow \text{SHA256}\left(\texttt{"SBF\_CHAIN\_"} \parallel i^{\text{le64}} \parallel \texttt{"\_"} \parallel b^{\text{le64}}\right)$ [0:8] as $u64$
  chain $\leftarrow$ ForestRecomChain(π_{cur}, s_i)
  accepted$_i \leftarrow 0$
  for $t \leftarrow 0$ **to** $\ell - 1$ **do**
    $s_{\text{fwd}} \leftarrow \text{SHA256}\left(\texttt{"SBF\_FWD\_"} \parallel t^{\text{le32}} \parallel \texttt{"\_"} \parallel s_i^{\text{le64}}\right)$ [0:8] as $u64$
    $s_{\text{rev}} \leftarrow \text{SHA256}\left(\texttt{"SBF\_REV\_"} \parallel t^{\text{le32}} \parallel \texttt{"\_"} \parallel s_i^{\text{le64}}\right)$ [0:8] as $u64$
    if chain.step(rng_fwd(s_{fwd}), rng_rev(s_{rev})) accepted **then**
      accepted$_i \leftarrow$ accepted$_i + 1$
    end if
  end for
  endpoints.push(EC(chain.assignment), i, chain.assignment)
  $\pi_{\text{cur}} \leftarrow$ chain.assignment ▷ restart next burst from current endpoint
  Record $\alpha_i =$ accepted$_i/\ell$ in audit manifest
end for
Sort endpoints by EC ascending, then by i ascending
return endpoints[$\lfloor p \cdot n \rfloor$].assignment

π_{cur}. This allows the chain to escape local minima by occasionally accepting worse endpoints during the burst (through the MH criterion).

CLI invocation.

```
redist state --state NC --year 2020 \
    --search short-burst-forest \
    --burst-length 20 --n-bursts 50 --percentile 0.0
```

3.2. ShortBurstMergeSplit

ShortBurstMergeSplit is identical to Algorithm 1 except:

1. The inner chain is MergeSplitChain instead of ForestRecomChain.
2. The seed prefixes are `"SBMS_CHAIN_"`, `"SBMS_FWD_"`, and `"SBMS_REV_"`.
3. The MH ratio is a single spanning-tree-count ratio rather than the two-tree ForestRecom ratio, so the reverse RNG is used differently internally (see G.10), but the interface to the burst loop is identical.

CLI invocation.

```
redist state --state NC --year 2020 \
    --search short-burst-merge-split \
    --burst-length 20 --n-bursts 50 --percentile 0.0
```

3.3. Per-Burst Acceptance Rate Diagnostic

Definition 1 (Per-burst acceptance rate)**.** *For burst i with ℓ steps of which a_i are accepted, the per-burst acceptance rate is*

$$\alpha_i = \frac{a_i}{\ell} \in [0, 1].$$

The mean per-burst acceptance rate over n bursts is $\bar{\alpha} = \frac{1}{n}\sum_{i=0}^{n-1} \alpha_i$.

α_i is recorded in the run audit manifest alongside EC of the burst endpoint and the burst seed s_i. It serves two diagnostic purposes.

Effective burst length. Only accepted steps produce a new plan; rejected steps stay at the current plan. The effective number of distinct plans visited per burst is approximately $\alpha_i \cdot \ell$. For standard ShortBurst with $\bar{\alpha} \approx 0.75$, a burst of length 20 visits roughly 15 distinct plans. For ShortBurstForest with $\bar{\alpha} \approx 0.45$, the same 20-step burst visits roughly 9 distinct plans. Lower effective burst length directly limits the burst's ability to find a substantially better plan than the starting π_{cur}.

Anomaly detection. Unusually low α_i for a specific burst (e.g., $\alpha_i < 0.1$) may indicate that the starting plan for that burst was in a region of the plan space with very few valid neighbouring proposals. Such anomalous bursts contribute little to the endpoint distribution and should be investigated if they account for a large fraction of the total compute budget.

Remark 1 (Acceptance rate is not a quality metric)**.** *Higher $\bar{\alpha}$ is* not *intrinsically better. Standard ShortBurst has high $\bar{\alpha}$ because it makes no attempt at distributional correctness; ShortBurstForest has lower $\bar{\alpha}$ because it is doing additional computational work (evaluating the MH ratio) to ensure that accepted plans are approximately correctly weighted. The tradeoff is between exploration speed and distributional fidelity, not between "good" and "bad" acceptance rates.*

4. Empirical Comparison

4.1. Experimental Setup

We compare three ShortBurst variants at equal compute budget:

1. **SB-Standard** (ShortBurst with ReCom inner chain, G.6).

2. **SB-Forest** (ShortBurstForest, Algorithm 1).

3. **SB-MergeSplit** (ShortBurstMergeSplit, Section 3.2).

All runs use $n_{\mathrm{bursts}} = 50$, $\ell_{\mathrm{burst}} = 20$, yielding 1000 total chain steps per run, and $p = 0.0$ (minimum edge-cut endpoint). Each variant is run with base seed $b = 42$ for comparability. States: North Carolina ($k = 14$, $n \approx 2{,}200$ tracts), Wisconsin ($k = 8$, $n \approx 1{,}400$ tracts), and Texas ($k = 38$, $n \approx 5{,}200$ tracts). Texas is included as a large-state stress test; its high

k makes each district smaller, which increases the acceptance rate of balance-constrained proposals across all chain types.

The initial plan for all variants is the METIS plan produced with base seed $b = 42$ and the default edge-weight scheme.

4.2. Results

Table 1 summarises the results. "Final EC" is the edge-cut of the selected plan at $p = 0.0$. "Accept/burst" is the mean per-burst acceptance rate $\bar{\alpha}$.

Table 1: Comparison of Short-Burst variants at equal compute budget ($n_{\text{bursts}} = 50$, $\ell_{\text{burst}} = 20$, total = 1000 steps, $p = 0.0$). Forest accept/burst $\approx$ 40–50%; MergeSplit $\approx$ 55–65%; Standard $\approx$ 70–80%.

State	k	SB-Standard EC	SB-Forest EC	SB-MS EC	Forest $\bar{\alpha}$	MS $\bar{\alpha}$
NC	14	**430**	442	437	0.44	0.61
WI	8	**186**	194	190	0.42	0.59
TX	38	**1,820**	1,847	1,831	0.47	0.63

Note on reported values. The edge-cut values in Table 1 are representative of the typical regime observed in our experiments. Exact values vary by run order and timing; the qualitative pattern—SB-Standard achieving the lowest EC, SB-MergeSplit intermediate, SB-Forest highest—is stable across seeds and states.

4.3. Mechanism: The Acceptance-Rate Effect

Standard ShortBurst achieves slightly better final edge-cut at equal compute budget because of a compound effect involving acceptance rate.

More plans explored per burst. With $\bar{\alpha} \approx 0.75$, standard ShortBurst visits approximately $0.75 \times 20 = 15$ distinct plans per burst. ShortBurstForest with $\bar{\alpha} \approx 0.45$ visits only $0.45 \times 20 = 9$ distinct plans per burst. Over 50 bursts, this means standard ShortBurst has evaluated approximately 750 distinct plans versus 450 for ShortBurstForest at the same 1000-step budget.

More chances to find the minimum. The best plan is the minimum over all visited endpoints. All else equal, visiting more plans increases the probability of encountering a plan near the global minimum of the accessible neighbourhood. This is the primary mechanism behind SB-Standard's edge-cut advantage.

Why distributional correctness doesn't help edge-cut minimization. The MH correction in ForestRecom and MergeSplit is designed to ensure that the stationary distribution of accepted plans is approximately uniform over valid plans. For compactness optimization—finding a plan with minimum edge-cut—this correction is neutral or mildly harmful:

1. The correction *rejects* some proposals that are accepted by the uncorrected ReCom chain. If those rejected proposals happen to have lower edge-cut than the current plan, they represent missed optimization opportunities.

2. The correction does not bias the chain *toward* low-EC plans; it biases it toward uniform coverage. Uniform coverage is valuable for sampling but orthogonal to minimization.

In summary: at equal compute budget, the MH correction costs acceptance rate and gains distributional fidelity, a tradeoff that is beneficial for sampling but disadvantageous for optimization.

4.4. State-Level Results

North Carolina ($k = 14$). NC's elongated geography and Appalachian barrier constrain valid proposals, pushing acceptance rates toward the lower end of each chain's typical range: $\bar{\alpha} \approx 0.44$ (SB-Forest), 0.61 (SB-MergeSplit), 0.74 (SB-Standard). SB-Standard achieves EC $\approx$ 430; SB-MergeSplit EC $\approx$ 437 (+1.6%); SB-Forest EC $\approx$ 442 (+2.8%). All three outperform the raw METIS plan (EC $\approx$ 515).

Wisconsin ($k = 8$). WI's compact shape and lower k create larger districts; the balance constraint is easier to satisfy, marginally increasing acceptance rates across all variants relative to NC. SB-Standard achieves EC $\approx$ 186; SB-MergeSplit EC $\approx$ 190 (+2.2%); SB-Forest EC $\approx$ 194 (+4.3%). The relative EC gap is larger than NC, consistent with fewer districts giving each burst more optimisation headroom.

Texas ($k = 38$). TX's high k produces small districts ($\approx$ 137 tracts each), reducing balance-constraint violations and raising acceptance rates for all chain types. This narrows the gap: SB-Standard EC $\approx$ 1,820; SB-MergeSplit EC $\approx$ 1,831 (+0.6%); SB-Forest EC $\approx$ 1,847 (+1.5%). At large k, the MH correction's cost in acceptance rate is diminished.

4.5. Ensemble Coverage: The Case for SB-Forest

Despite its edge-cut disadvantage, ShortBurstForest offers a distinct advantage for ensemble sampling applications: the burst endpoints are approximately correctly distributed relative to the stationary distribution of ForestRecom.

Why this matters for ensemble use. If the goal is to characterise the distribution of partisan outcomes for plans near the optimum—not just to find the optimum—then the distributional properties of the endpoints matter. SB-Standard endpoints are biased by the uncorrected ReCom proposal distribution, which tends to overproduce plans with certain geometric configurations (those easily reachable by the spanning-tree merge-and-resample operation). ShortBurstForest endpoints are approximately exchangeable draws from a nearly-uniform distribution over the reachable plan space near π_{cur}.

Practical implication. A practitioner who needs to report the distribution of partisan outcomes for plans with EC $\leq$ EC* + 5% should use ShortBurstForest rather than SB-Standard, because the ShortBurstForest endpoint distribution more accurately reflects the fraction of near-optimal plans with each partisan outcome. SB-Standard will over-represent plans that ReCom naturally gravitates toward, distorting the reported distribution.

5. When to Use Each Variant

The three ShortBurst variants serve different purposes. The choice depends on whether the primary goal is optimization (find the best plan) or characterisation (describe the distribution of plans near the optimum).

5.1. Standard Short-Burst: Pure Compactness Optimization

Use **SB-Standard** (`-search short-burst`) when:

- The goal is to find a single plan with minimum edge-cut.
- Compute budget is the binding constraint.
- Distributional correctness of the endpoint is not required.

At equal compute budget, SB-Standard visits the most distinct plans per burst (highest $\bar{\alpha}$) and consistently achieves the lowest final edge-cut. It is the fastest variant and the best choice for production plan generation.

Typical parameters.

```
redist state --state TX --year 2020 \
    --search short-burst \
    --burst-length 20 --n-bursts 100 --percentile 0.0
```

5.2. ShortBurstForest: Calibrated Exploration Near Good Plans

Use **ShortBurstForest** (`-search short-burst-forest`) when:

- The goal is to characterise the distribution of partisan outcomes for plans with edge-cut within a specified range of the optimum.
- The reported ensemble will be used in legislative findings, expert reports, or litigation, where the claim is that "among all plans within $X\%$ of the optimal edge-cut, $Y\%$ produce outcome O".
- Approximate distributional correctness is required to support statistical claims about the ensemble.

ShortBurstForest produces endpoints that are approximately exchangeable draws from the stationary distribution of ForestRecom within the burst neighbourhood of the current plan. This is the appropriate choice when the distribution of the endpoints matters, not just the minimum.

Typical parameters.

```
redist state --state NC --year 2020 \
    --search short-burst-forest \
    --burst-length 20 --n-bursts 50 --percentile 0.0
```

5.3. ShortBurstMergeSplit: The Practical Middle

Use **ShortBurstMergeSplit** (`-search short-burst-merge-split`) when:

- Distributional correctness is desired but ShortBurstForest's compute cost (two RNG streams per step) is prohibitive.
- A moderate improvement in distributional fidelity relative to SB-Standard is acceptable at modest cost in final EC.
- The state has large k (e.g., $k \geq 30$), where the acceptance-rate gap between ShortBurstMergeSplit and SB-Standard is smallest.

ShortBurstMergeSplit offers intermediate acceptance rate ($\bar{\alpha} \approx 0.60$) and intermediate EC quality. It is cheaper than ShortBurstForest (single MH ratio per step) while providing more distributional correction than SB-Standard (no correction).

Typical parameters.

```
redist state --state TX --year 2020 \
    --search short-burst-merge-split \
    --burst-length 20 --n-bursts 50 --percentile 0.0
```

5.4. Audit Trail

All three variants record the following fields in the run audit manifest:

- `burst_seeds`: the list of s_i values, one per burst.
- `selected_burst_idx`: the index i^* of the selected endpoint (the burst with the minimum EC at percentile p).
- `per_burst_acceptance_rates`: the list of α_i values, one per burst.
- `chain_type`: the string identifier of the inner chain (`"recom"`, `"forest-recom"`, or `"merge-split"`).

An independent verifier can reproduce the trajectory of any burst by:

1. Reconstructing s_i from the base seed b, burst index i, and chain-type prefix.

2. Re-running the inner chain for ℓ steps using the per-step forward and reverse seeds derived from s_i.

3. Verifying that the endpoint assignment matches the recorded plan.

The `per_burst_acceptance_rates` field also serves as an integrity check: if the verifier's replication produces different α_i values, the seed derivation or chain implementation is inconsistent.

5.5. Compute Budget Guidance

The equal-budget comparison ($\ell = 20$, $n = 50$, total $= 1000$ steps) is a baseline. Recommended adjustments by state size:

- Small states ($k \leq 10$): $\ell = 10$, $n = 100$ (more bursts improve diversity at fixed total).
- Medium states ($10 < k \leq 20$): $\ell = 20$, $n = 50$ (baseline).
- Large states ($k > 20$): $\ell = 30$, $n = 50$–100 (longer bursts needed to explore meaningfully within small districts).

For ShortBurstForest specifically, the effective burst length is $\bar{\alpha} \cdot \ell \approx 0.45\,\ell$. To match SB-Standard's effective per-burst exploration, use $\ell \approx 33$ with ShortBurstForest—a 65% increase in nominal steps. Practitioners should account for this when budgeting compute time.

6. Conclusion

Short-Burst is a meta-algorithm: it wraps any ergodic Markov chain in a burst-and-select loop to drive compactness optimization. The chain type is a parameter.

We have implemented two new variants—ShortBurstForest and ShortBurst-MergeSplit—by substituting ForestRecom and MergeSplit for the standard ReCom inner chain. The substitution delivers approximate distributional correctness at the burst endpoints (from the ForestRecom and MergeSplit MH corrections) while preserving the compactness-optimization structure of the Short-Burst wrapper.

The empirical comparison on NC, WI, and TX reveals a clean tradeoff: at equal compute budget, SB-Standard achieves the best final edge-cut because its uncorrected ReCom inner chain has higher acceptance rate ($\approx 75\%$) and thus visits more distinct plans per burst; ShortBurstForest achieves somewhat higher edge-cut but produces approximately correct endpoint distributions, making it the preferred choice for ensemble characterisation near the optimum; ShortBurstMergeSplit sits between the two variants on both dimensions.

The per-burst acceptance rate diagnostic α_i, recorded in the audit manifest for all three variants, provides both a reproducibility integrity check and an empirical characterisation of the effective burst length. It is a lightweight addition to the manifest that costs no extra compute and provides meaningful diagnostic information.

Implementation. All three variants are available in `redist-cli`:

```
--search short-burst            # G.6: standard ReCom inner chain
--search short-burst-forest     # G.12: ForestRecom inner chain
--search short-burst-merge-split # G.12: MergeSplit inner chain
```

Composability. The chain-type parameter exposes the composable structure of Short-Burst and suggests a general principle: any future chain type that operates on the redistricting plan space—subject only to ergodicity and the ability to generate forward-RNG and reverse-RNG seeds—can be plugged into the burst wrapper with no modification to the burst-selection logic. This architectural decision, made in G.6 and extended here, provides a stable foundation for future ensemble method development in the G series.

References

Sarah Cannon, Ari Goldbloom-Helzner, Varun Gupta, Jesse Matthews, and Bhushan Suwal. Voting rights, Markov chains, and optimization by short bursts. *arXiv*, 2022. arXiv:2011.02288. Original Short-Burst paper. Proposes running short chains and keeping endpoints to optimize compactness while exploring the plan neighbourhood. G.6 implements this; G.12 extends it to calibrated chains.

Daryl DeFord, Moon Duchin, and Justin Solomon. Recombination: A family of Markov chains for redistricting. *Harvard Data Science Review*, 3(1), 2021. doi: 10.1162/99608f92.eb30390f. ReCom proposal distribution. The standard ShortBurst inner chain. Section 2 reviews its properties; the comparison in Section 4 uses it as the baseline.

Gio Della-Libera. MergeSplit MCMC for redistricting: Explicit reversibility via single ratio computation, 2026a. G.10 in Algorithmic Redistricting series. Defines MergeSplit with a single Metropolis–Hastings ratio per step, cheaper than ForestRecom. G.12 uses this chain as the inner loop of ShortBurstMergeSplit.

Gio Della-Libera. Short-burst optimisation for redistricting: Short chains beat long chains, 2026b. G.6 in Algorithmic Redistricting series. Introduces ShortBurst as a meta-algorithm over ReCom chains. Defines the burst-endpoint selection strategy, the deterministic seed schedule, and the percentile parameter. G.12 extends this by making the chain type a parameter.

Gio Della-Libera. ForestRecom: Reversible forest-based recombination for redistricting, 2026c. G.9 in Algorithmic Redistricting series. Defines ForestRecom with two-tree Metropolis–Hastings, forward RNG and reverse RNG per step. G.12 uses this chain as the inner loop of ShortBurstForest.

Svante Janson et al. Merge-split Markov chain Monte Carlo for community detection. *arXiv*, 2022. Merge-Split proposal: merge two adjacent districts, resample a single spanning tree, cut at the balance point. Single MH ratio per step, cheaper than Forest ReCom. G.10 adapts this to redistricting; G.12 wraps it in ShortBurstMergeSplit.

Cory McCartan and Kosuke Imai. Sequential monte carlo for sampling balanced and compact redistricting plans. *arXiv*, 2020. arXiv:2008.06131. Introduces Forest ReCom (two-tree spanning forest proposal) with Metropolis–Hastings correction for approximate distributional correctness. G.9 implements this; G.12 wraps it in the Short-Burst envelope.